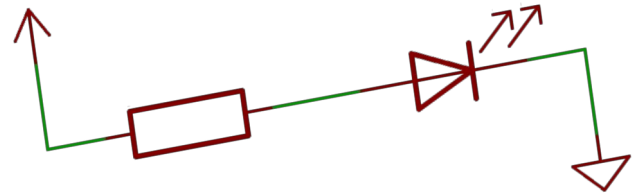


Schematic Design



Starting guide

Intro

Schematic design is one of the core skills you will need in most, if not all, forms of engineering. In this case, we will focus on *electrical engineering*. Although this guide was created with PCB design in mind, this advice should be useful for most electrical engineering.

Schematic design is arguably one of the most important parts of electrical engineering, not only for your sake, but for everyone else's sake. Good schematic design is all about *readability*! That means mistakes are easier to correct, your work is more organized for future reference, and other engineers can easily assist you with any problem you may have. Good schematic design will reduce the chances of needing to respin boards, it'll make work a breeze, and *it'll impress potential employers!* Everyone wins!

To start off this guide, we'll assume you know basic electronic concepts such as KVL, KCL, Ohm's law, etc. We'll also assume you are familiar with transistors, diodes, resistors, and capacitors. This guide is meant to be read before you begin PCB layout.

In this guide, we will use KiCad to illustrate our schematics. Generally, though, most rules of schematic making are universal- it does not matter what program or platform you use.

Starting Off

As mentioned before, this guide will use KiCad to demonstrate schematic design.

Several tools are available for free, including KiCad, Falstad Circuit Simulator, LTSpice, EasyEDA, CircuitMaker, LibrePCB, and more.

In this guide, we will be using generic components for the most part. Any advice given here is not meant to be specific to any one IC or part, applications vary- this is just meant to start you out with general good practice to build on with experience.

With that out of the way, let's move on to our rules...

Do's, Don'ts

Let's start off with some simple rules, and follow up with some examples.

Do:

- Point power flags **Up**, and ground **Down**.
- Use **90 Degree angles!**
- Use **contrasting colours** for overlapping wires
- Use net labels, and net classes!
- Arrange your circuits so that they flow from left to right, up to down, following conventional current.
- Group parts of your schematic by function- Generally, as well as going up-down, left-right, you want to separate parts of your circuits into logical groups. We'll get more into this later.
- Label components *clearly and concisely*, and use symbols appropriate for your component!

Don't:

- Put text inside of symbols!
- Point ground **Up** or power flags **Down!**
- Draw wires through symbols- this will make your schematic illegible very quickly
- Cross wires excessively for no real reason- only ever cross your wires if you **MUST!**
- Use angles other than 90 degrees! That means no 45, 32, 78 degree angles... *Only 90!*
- Arrange your circuit so that it is the same scale as the real thing, unless it would make your schematic more concise!
- Make things up if you are completely unsure about what you're doing (*trust me, you'll thank yourself...*)

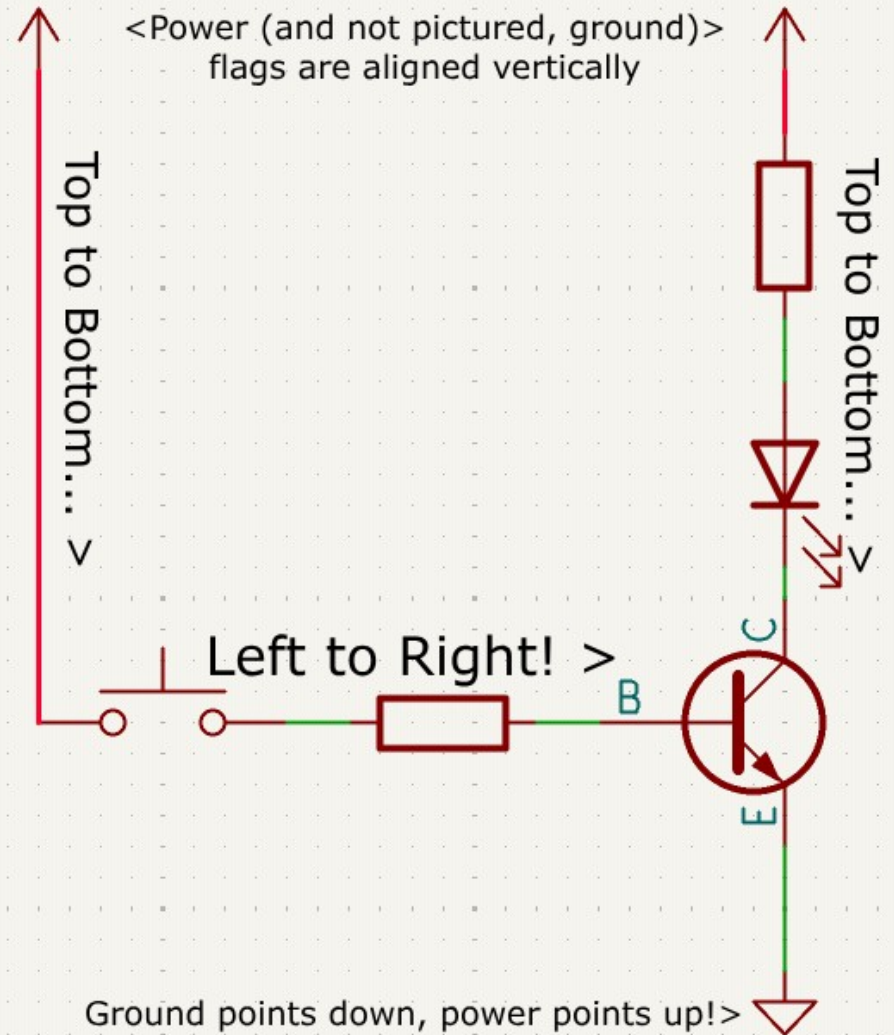
Good practice

Let's get a feel for the “Feng Shui” of schematic design.

Even without any reference designators, you can probably tell what's going on, right?

The circuit has a sort of “flow” to it... Any engineer having a simple glance at this schematic could tell what it does, no mistakes to be made. This schematic has intent, and good practice.

The colours of the wires do not matter right now; the point is that you can identify this circuit without much difficulty. It flows from “input” to “output”, “positive” to “ground”.



Bad practice

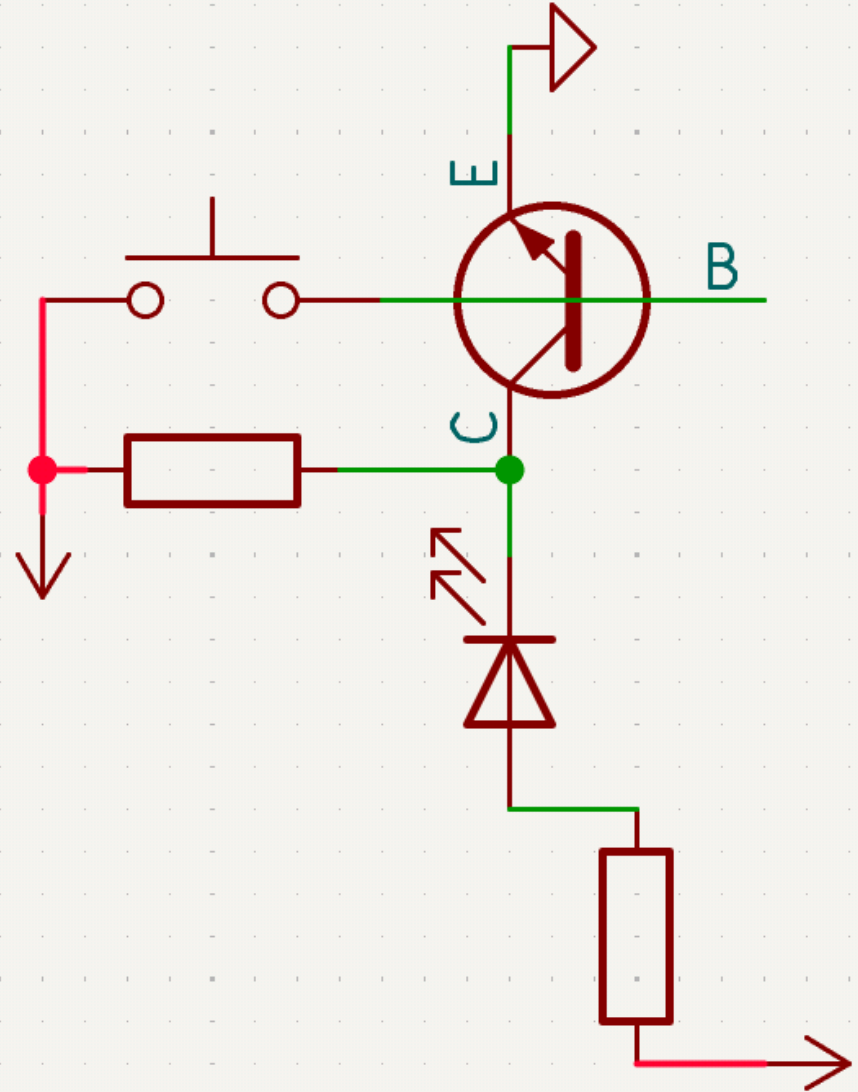
Now, let's introduce our evil twin here, so to speak.

Could you effectively tell what is going on in this circuit, at all?

Maybe someone wired it on their own time, but to anybody else (and even to them in the future), it does not make much sense at all.

Could you tell that this schematic has a few mistakes? It'd be a little difficult!

Current doesn't appear to flow any certain way, nor can you tell what does what, and the parts are all jumbled up together.

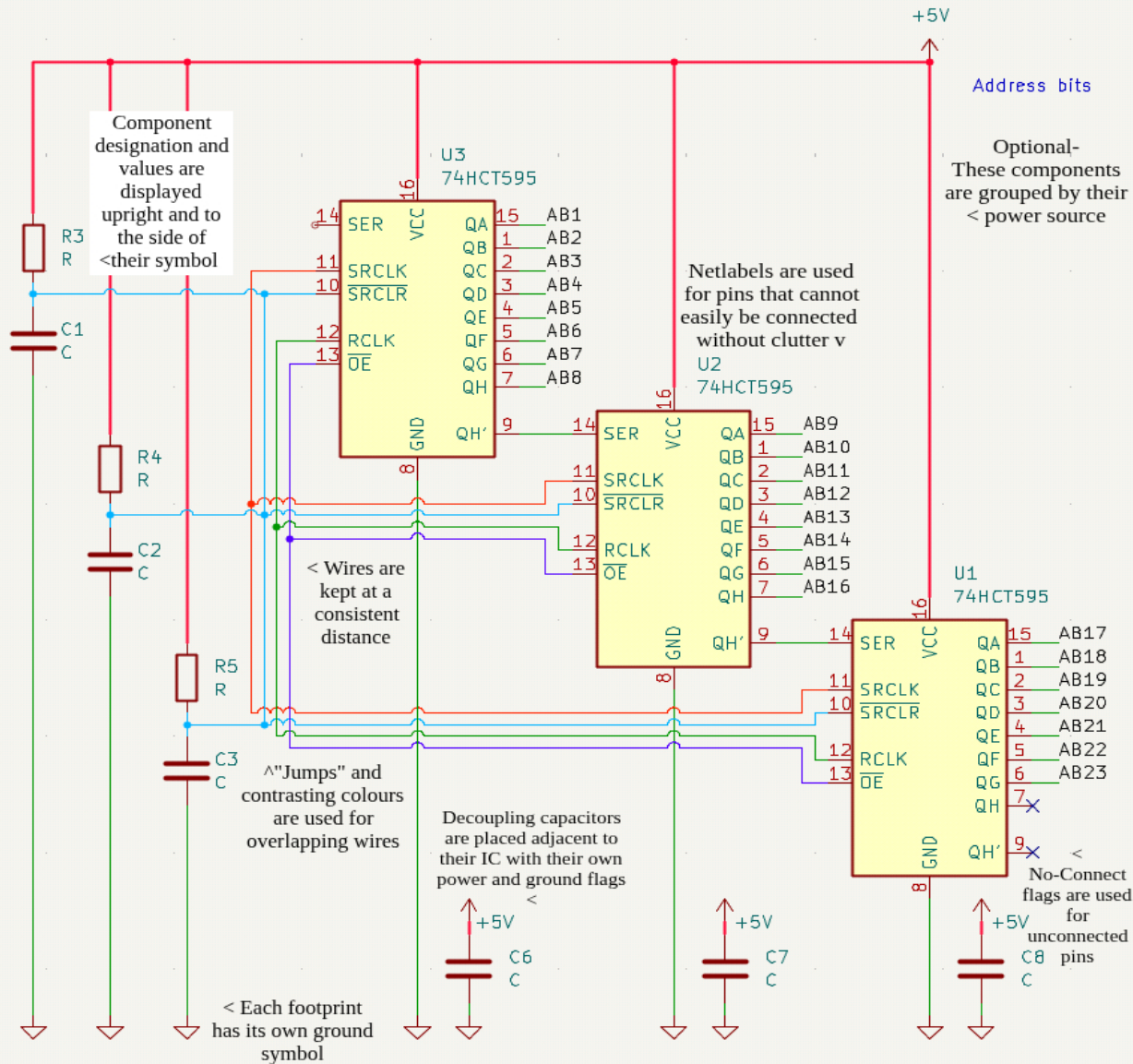


More complex now...

Now, let's move on to something more complex. Here is a new schematic, now with specific parts and real implementations!

You'll notice that although there are overlapping wires, their purpose is hard to confuse, and they are each identifiable. You can tell what each thing does, and it's easy to read the flow- these shift registers cascade to one another, writing to 23 different lines.

Notice the net labels with no wires- wiring these components directly would create an immense amount of clutter, so we simplified it!

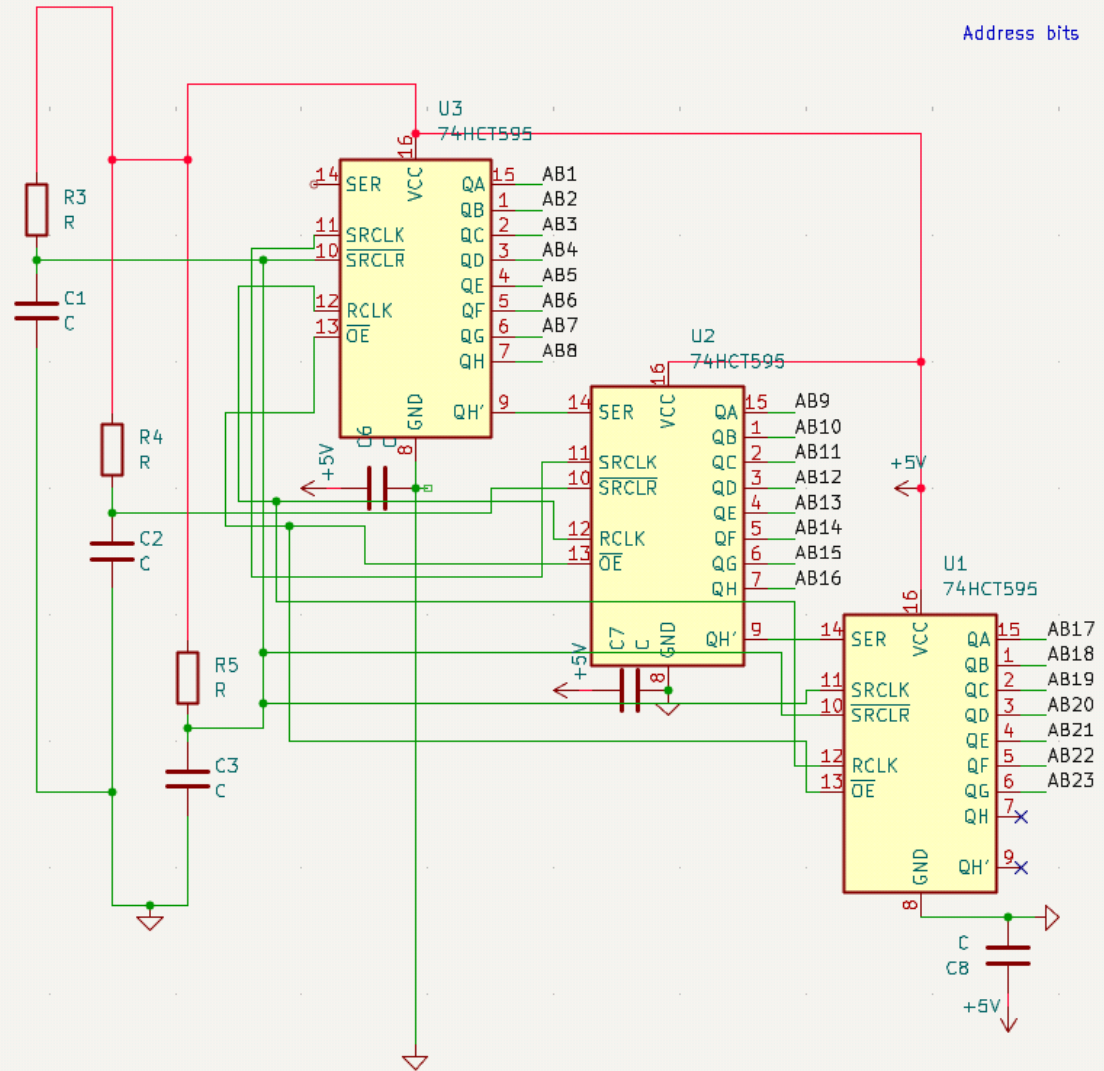


But what if...

Now, let's toss everything on here again and connect wires to maybe the right points.

You can't really tell what's going on, now. Even though this is the same circuit, functionally, as the last, it is illegible. Anybody assisting you would be unable to draw any conclusion aside from "well, it looks messy".

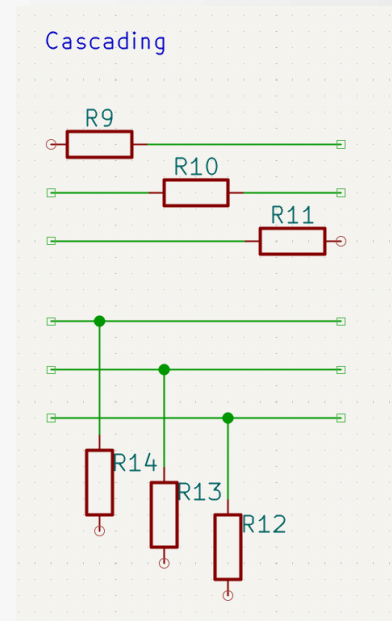
Even you could not debug it- it's hard to tell where one trace begins and where the other stops. Suppose I introduced a mistake here, would you be able to spot it easily?



Tips

Alright, now let's move on to some tips!

- Rotate your reference designators so that they are upright, and move them closer to the symbol so that they are not taking up excessive space, be sure to avoid clutter, though!
- Use net labels instead of wires when dealing with crowded groups of wires or pins arranged in an unconventional order
- Cascade components (see image>)
- Annotate your schematic with text/images (platform dependent)
- Use hierarchical sheets (platform dependent, but most should have something along those lines)



GL;HF!

If any part of this guide is
dissatisfactory or incomplete, feel
free to contact me @spheresva on
most platforms (or spheres.va on
discord). If you like my work, any
sort of support is appreciated!